



UNIVERSITÉ
CAEN
NORMANDIE

Analyse des algorithmes de tri

Conception Logicielle 3

THOMAS Matthieu : 22304534

SIAGHI Massinissa : 22312276

MARIE Vianney : 22106226

TELLIER Basile : 22104032

2 avril 2026

Table des matières

1	Introduction	3
1.1	Présentation générale du projet	3
1.2	Problématique	3
1.3	Plan du rapport	3
2	Structuration du projet	4
2.1	Analyse des besoins	4
2.2	Fonctionnalités implémentées	4
2.3	Organisation du travail	5
3	Éléments techniques	6
3.1	Structures de données utilisées	6
3.2	Patterns de conception	6
3.2.1	Observer	6
3.2.2	Template Method	7
3.2.3	Strategy	7
3.2.4	Facade	7
3.2.5	MVC	7
4	Architecture du projet	8
4.1	Vue d'ensemble	8
4.2	Couche Modèle	8
4.2.1	Génération des tableaux	8
4.2.2	Hiérarchie des algorithmes de tri	9
4.2.3	SortingModel	10
4.3	Couche Contrôleur	10
4.4	Couche Vue	10
4.4.1	Structure générale de la vue	10
4.4.2	Graphique en barres : ArrayBarChart	11
4.4.3	Panneau de contrôle : ControlPanel	11
5	Expérimentations et usages	12
5.1	Expérimentation	12
5.1.1	Méthode de fonctionnement	12
5.1.2	Lancement de l'experimentation	12
5.1.3	Génération des résultats	12
5.1.4	Aggregation des résultats	13
5.2	Analyse des résultats	13
5.2.1	Lecture et traitement des données	13
5.2.2	Génération des courbes	14
6	Réponse à la problématique	15
6.1	Comparaisons	15
6.1.1	Hiérarchie générale	15
6.1.2	Influence de la localisation du désordre	16
6.2	Accès	18
6.2.1	Hiérarchie générale	18
6.2.2	Influence de la localisation du désordre	19
6.3	Swap	20
6.3.1	Hiérarchie générale	20
6.4	Temps d'exécution	21

6.4.1	Hiérarchie générale	21
6.5	Synthèse par algorithme	21
6.6	Conclusion	22

1 Introduction

1.1 Présentation générale du projet

Ce projet s'inscrit dans le cadre de l'UE *Conception Logicielle 3*. Il consiste à répondre à une **question scientifique** autour des algorithmes de tri, structurée en quatre grandes étapes :

- **Modélisation** : conception de l'architecture logicielle
- **Génération** : production de tableaux paramétrés
- **Expérimentation** : exécution des algorithmes et collecte des métriques
- **Analyse** : interprétation des résultats

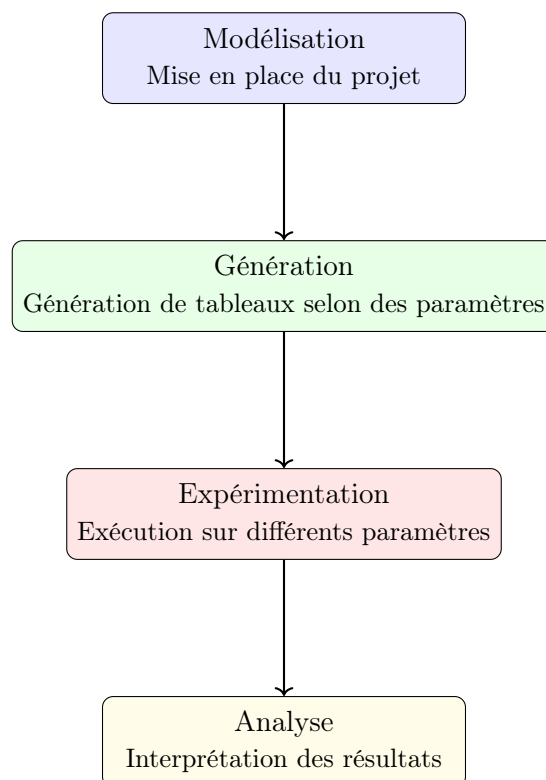


FIGURE 1 – Les quatre grandes étapes du projet

1.2 Problématique

La problématique centrale est la suivante : **comment les paramètres d'entrée d'un tableau (taille, degré et zone de désordre) influencent-ils les performances des algorithmes de tri en termes de comparaisons, d'accès mémoire, d'échanges et de temps d'exécution ?**

Pour répondre à cette question, nous avons développé une application de visualisation interactive permettant d'observer le déroulement de chaque algorithme en temps réel, et un module d'expérimentation permettant de collecter des métriques sur de grands volumes de données.

1.3 Plan du rapport

- **Section 2 – Structuration** : besoins fonctionnels, fonctionnalités implémentées, organisation du travail.
- **Section 3 – Éléments techniques** : structures de données, patterns de conception utilisés.
- **Section 4 – Architecture** : description de l'architecture MVC, diagrammes UML des principales couches.
- **Section 5 – Expérimentations** : résultats obtenus et analyse comparative des algorithmes.
- **Section 6 – Conclusion** : bilan et axes d'amélioration.

2 Structuration du projet

2.1 Analyse des besoins

- **Génération de données :**
 - Génération de tableaux d'entiers de taille et de degré de désordre paramétrables.
 - Support de quatre zones de désordre : aléatoire, début, milieu, fin.
- **Algorithmes de tri :**
 - Implémentation de cinq algorithmes : Bubble Sort, Insertion Sort, Quick Sort, Merge Sort, Counting Sort.
 - Instrumentation de chaque opération élémentaire (lecture, écriture, comparaison, échange) pour la collecte de métriques.
- **Visualisation interactive :**
 - Interface graphique Swing affichant le tableau sous forme de barres colorées selon l'opération en cours.
 - Contrôles de démarrage, pause, reset, et réglage de la vitesse.
 - Affichage en temps réel des métriques (comparaisons, accès, échanges, temps).
- **Architecture logicielle :**
 - Respect strict du patron MVC pour la séparation des responsabilités.
 - Code modulaire, extensible et documenté.

2.2 Fonctionnalités implémentées

- Génération de tableaux avec quatre modes de désordre configurable.
- Cinq algorithmes de tri entièrement instrumentés.
- Visualisation en barres avec code couleur par opération : orange (comparaison), vert (échange), bleu (lecture), cyan (écriture).
- Panneau de métriques mis à jour en temps réel.
- Contrôle fin de l'exécution : démarrage, pause/reprise, reset, nouveau tableau, vitesse réglable de 10 à 500 ms par opération.
- Module d'expérimentation hors interface graphique pour les benchmarks.

2.3 Organisation du travail

Membre	Responsabilités principales
Massinissa	Conception de l'architecture MVC et de l'interface graphique : — Interface Sort et classe abstraite AbstractSort (instrumentation, métriques, Template Method) — Implémentation de MergeSort et CountingSort — Mise en place du patron MVC complet — Développement de toute la vue Swing (Main-Frame, panneaux, graphique en barres, métriques) — Diagrammes UML — Rapport — Soutenance finale
Vianney	Génération des données : — Classe Generator avec les quatre modes de désordre — Paramétrage de la taille et du pourcentage de désordre — Générateur avec validation — Générateur avec seed
Matthieu	Analyse — Implémentation de InsertionSort et BubbleSort — Soutenance de mi-rendu — Création des courbes avec matplotlib — Analyse des comportement des courbes — Rapport — Soutenance finale
Basile	Expérimentation et rédaction : — Implémentation de Quicksort — Génération des données — Placement dans CSV — Agrégation des données — Rapport — Soutenance finale

TABLE 1 – Répartition des rôles au sein de l'équipe

3 Éléments techniques

3.1 Structures de données utilisées

- **Tableau d'entiers** (`int[]`) : structure principale manipulée par tous les algorithmes. Les opérations sont effectuées en place pour limiter la consommation mémoire.
- `List<Integer>` : utilisée par le générateur pour mélanger des sous-segments du tableau via `Collections.shuffle`.
- **Tableau auxiliaire** (`int[]`) : utilisé par `MergeSort` lors de la phase de fusion, et par `CountingSort` pour le tableau de comptes.
- `SwingWorker<Void,Void>` : structure de contrôle asynchrone permettant d'exécuter le tri dans un thread d'arrière-plan sans bloquer l'interface graphique (EDT).

3.2 Patterns de conception

3.2.1 Observer

Le modèle `SortingModel` hérite de `AbstractModeleEcoutable` qui maintient une liste d'`EcouteurModele`. À chaque opération de tri, `updateVisualization()` appelle `fireChangement()` qui notifie tous les composants abonnés (`ArrayBarChart`, `MetricsDisplay`, `VisualizationPanel`). Cela garantit que la vue se rafraîchit automatiquement sans couplage direct avec le modèle.

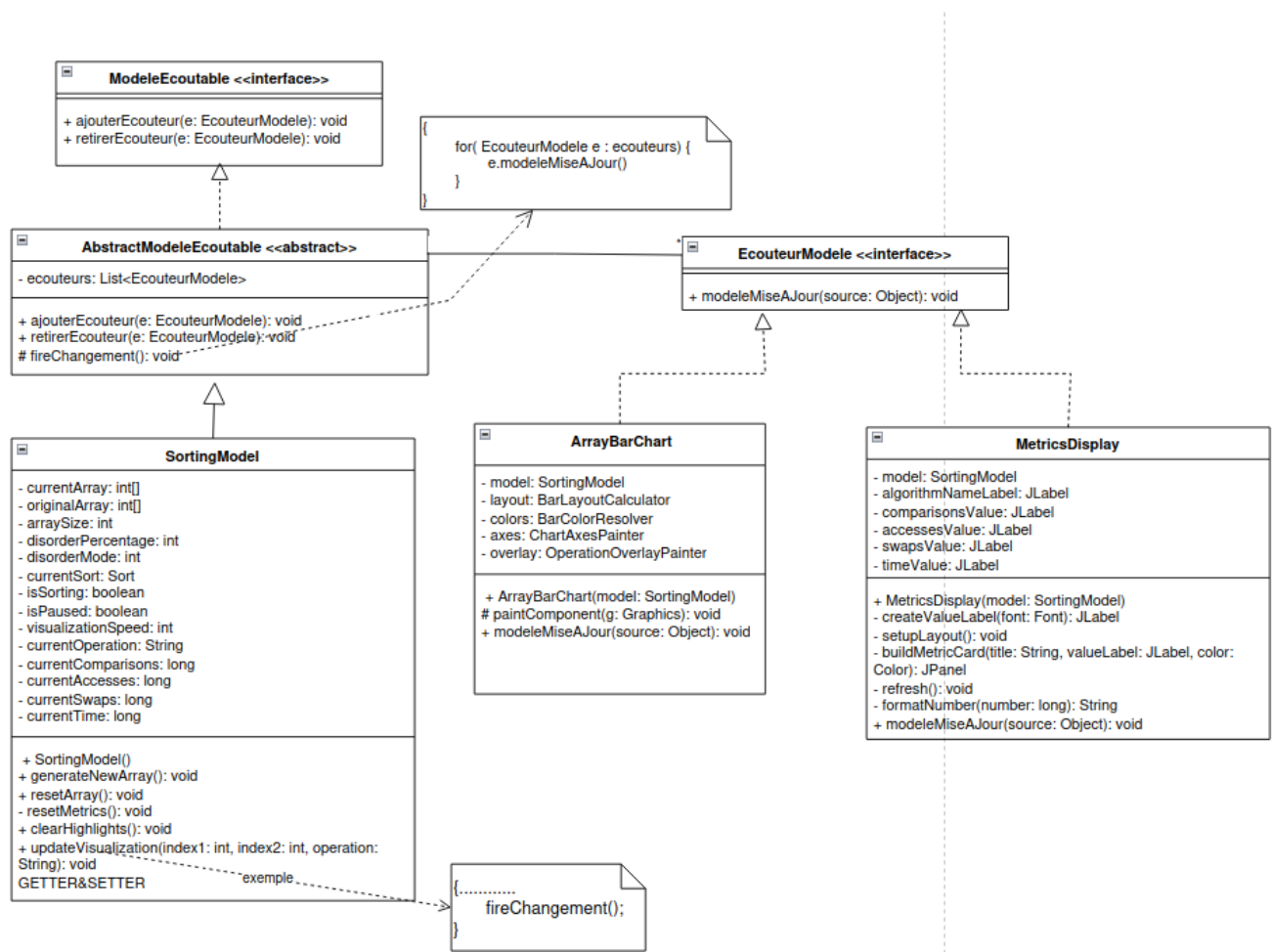


FIGURE 2 – Diagramme UML du pattern Observer

3.2.2 Template Method

La classe abstraite `AbstractSort` définit le squelette du tri dans la méthode finale `sort()` : réinitialisation des métriques, démarrage du chronomètre, délégation à `sortImpl()`, puis enregistrement du temps écoulé. Chaque sous-classe implémente uniquement `sortImpl()` avec sa logique propre, sans pouvoir altérer le comportement commun.

```
public final void sort(int[] array) {
    resetMetrics();
    this.startTime = System.nanoTime();
    sortImpl(array);
    this.timeNano = System.nanoTime() - this.startTime;
}

protected abstract void sortImpl(int[] array);
```

Listing 1 – Template Method dans `AbstractSort`

3.2.3 Strategy

L'interface `Sort` joue le rôle de stratégie. `SortingModel` référence uniquement un `Sort`, ce qui permet de substituer l'algorithme à l'exécution via `setCurrentSort()` sans modifier aucun autre composant. Ajouter un nouvel algorithme revient à créer une classe qui étend `AbstractSort`, aucune modification du code existant n'est nécessaire (principe Open/Closed).

3.2.4 Facade

`SortingController` expose une interface simplifiée à la vue : `startSorting()`, `resetArray()`, `togglePause()`, etc. Les panneaux de contrôle ne connaissent ni `SortingModel` ni `SwingWorker` : toute la complexité de l'orchestration asynchrone est encapsulée dans le contrôleur.

```
// Ce que voit la vue : une interface simple
controller.startSorting();
controller.resetArray();
controller.togglePause();
controller.setArraySize(100);

// Ce que le controleur cache a la vue :
private void stopCurrentSort() {
    model.setPaused(false);
    model.setIsSorting(false);
    if (currentWorker != null) {
        currentWorker.cancel(true); // interruption du thread
        currentWorker = null;
    }
}
```

Listing 2 – Facade : `SortingController` simplifie l'accès au modèle

3.2.5 MVC

L'architecture suit strictement le patron Modèle-Vue-Contrôleur : `SortingModel` contient l'état et la logique, la vue (`VisualizationPanel` et ses sous-composants) observe et affiche, et `SortingController` traduit les actions utilisateur en appels au modèle sans que la vue et le modèle ne se connaissent directement.

4 Architecture du projet

4.1 Vue d'ensemble

Le projet est structuré en trois packages principaux : `modele`, `controleur` et `vue`, conformément au patron MVC. Le package `modele` est lui-même subdivisé en `modele.sorting` (algorithmes), `modele.generateur` (génération des tableaux) et `modele.event` (infrastructure Observer).

4.2 Couche Modèle

4.2.1 Génération des tableaux

La classe `Generator` produit des tableaux d'entiers $[0, 1, \dots, n-1]$ initialement triés, puis partiellement mélangés selon un pourcentage et un mode de désordre (1 = aléatoire, 2 = début, 3 = milieu, 4 = fin). La méthode statique `generate()` est le point d'entrée unique utilisé par `SortingModel`.

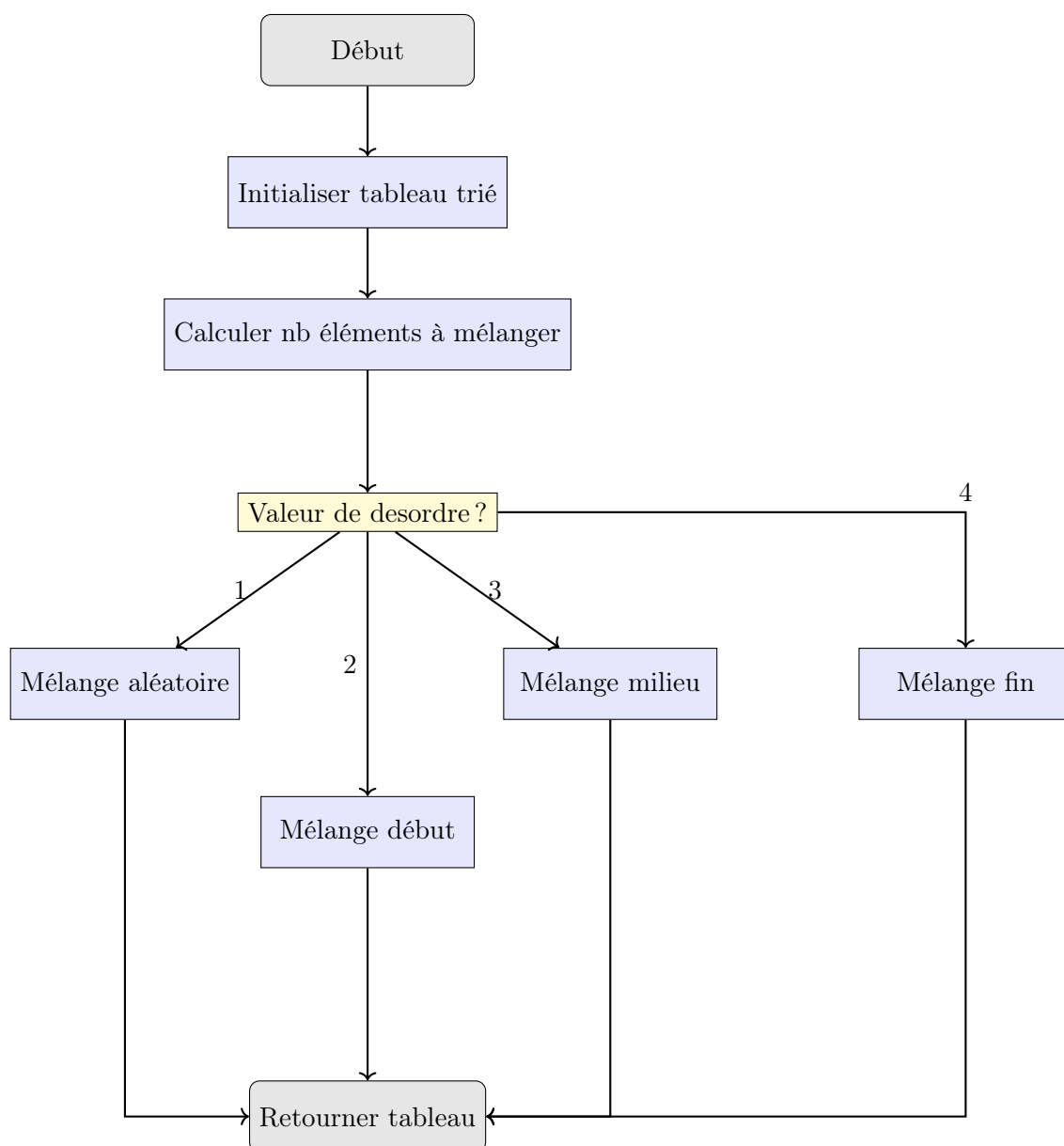


FIGURE 3 – Organigramme de la fonction `generateurTab`

Le mélange des éléments du tableau est réalisé à l'aide de la méthode `MelangeurShuffle`, qui fait appel à `Collections.shuffle()`. Cette fonction de la bibliothèque standard Java permet d'obtenir un mélange aléatoire fiable et uniforme des éléments d'une liste.

Une version alternative du générateur a également été développée en utilisant une *seed* (de type `Integer`) pour le générateur aléatoire. Cela permettait de reproduire exactement les mêmes mélanges d'une exécution à une autre, ce qui est utile pour les tests.

Par ailleurs, une fonction `Validator` avait été mise en place. Celle-ci prenait en argument `int[] tab`, `int taille` et `int nombreAleatoire`, et avait pour objectif de vérifier que le tableau était bien mélangé conformément aux attentes (nombre d'éléments modifiés, position des éléments, etc.).

Ce validateur était uniquement utilisé lorsque la *seed* était nulle, afin de contrôler le caractère correctement aléatoire du mélange. Cependant, cette version n'étant pas utilisée dans la version finale du projet, ces éléments n'ont pas été intégrés au rendu final.

4.2.2 Hiérarchie des algorithmes de tri

L'interface `Sort` définit le contrat commun à tous les algorithmes. `AbstractSort` en fournit l'implémentation de base : instrumentation des opérations (`read`, `write`, `swap`, `isLess`), gestion des métriques et vérification de l'interruption du thread à chaque opération pour permettre l'annulation propre du tri.

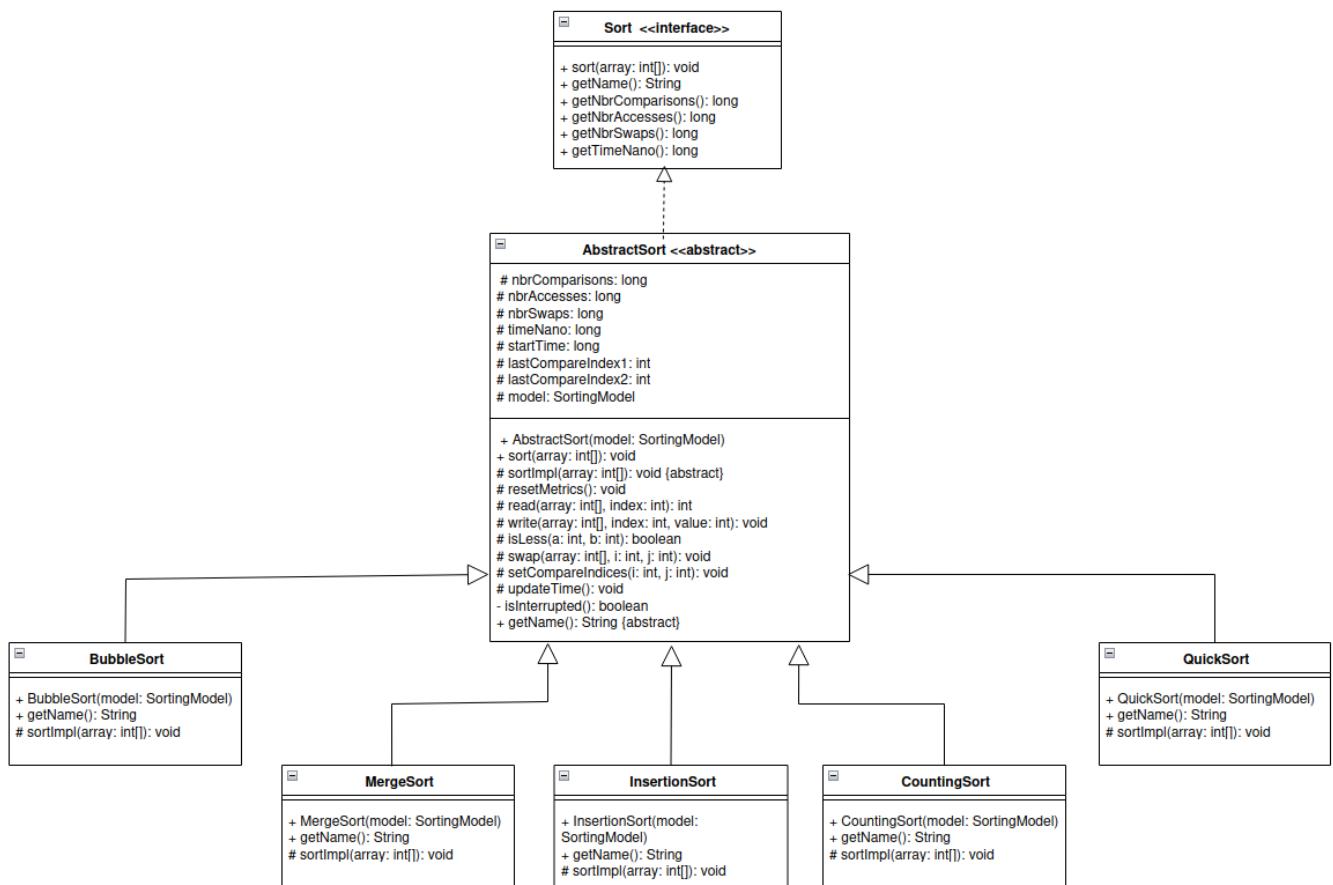


FIGURE 4 – Hiérarchie des algorithmes de tri

Algorithme	Complexité moyenne	En place	Particularités
BubbleSort	$O(n^2)$	Oui	Arrêt anticipé si aucun échange
InsertionSort	$O(n^2)$	Oui	$O(n)$ sur tableau presque trié
QuickSort	$O(n \log n)$	Oui	Pivot aléatoire, non stable
MergeSort	$O(n \log n)$	Non	Stable, tableau auxiliaire $O(n)$
CountingSort	$O(n + \max(tab))$	Non	Non comparatif, entiers positifs uniquement

TABLE 2 – Comparaison des algorithmes implémentés

4.2.3 SortingModel

SortingModel est le cœur de l'application. Il centralise l'état du tableau, les paramètres de génération, le contrôle de l'exécution (pause, vitesse) et les métriques en temps réel. La méthode `updateVisualization()` est appelée par l'algorithme à chaque opération élémentaire : elle met à jour les indices mis en évidence, synchronise les métriques, notifie les vues via `fireChangement()`, gère la pause (boucle d'attente) et applique le délai de visualisation.

4.3 Couche Contrôleur

SortingController instancie les cinq algorithmes et gère le cycle de vie du tri. La méthode privée `stopCurrentSort()` libère la pause, marque le tri comme terminé et interrompt le **SwingWorker** courant via `cancel(true)` — ce qui propage une interruption dans le thread de tri, stoppant toutes les opérations instrumentées grâce à la vérification `Thread.currentThread().isInterrupted()` présente dans chaque méthode d'**AbstractSort**.

4.4 Couche Vue

4.4.1 Structure générale de la vue

La vue est organisée en deux niveaux de composition. **MainFrame** contient **VisualizationPanel**, qui agrège à son tour trois sous-composants : **ControlPanel** (contrôles), **ArrayBarChart** (graphique), et **MetricsDisplay** (métriques).

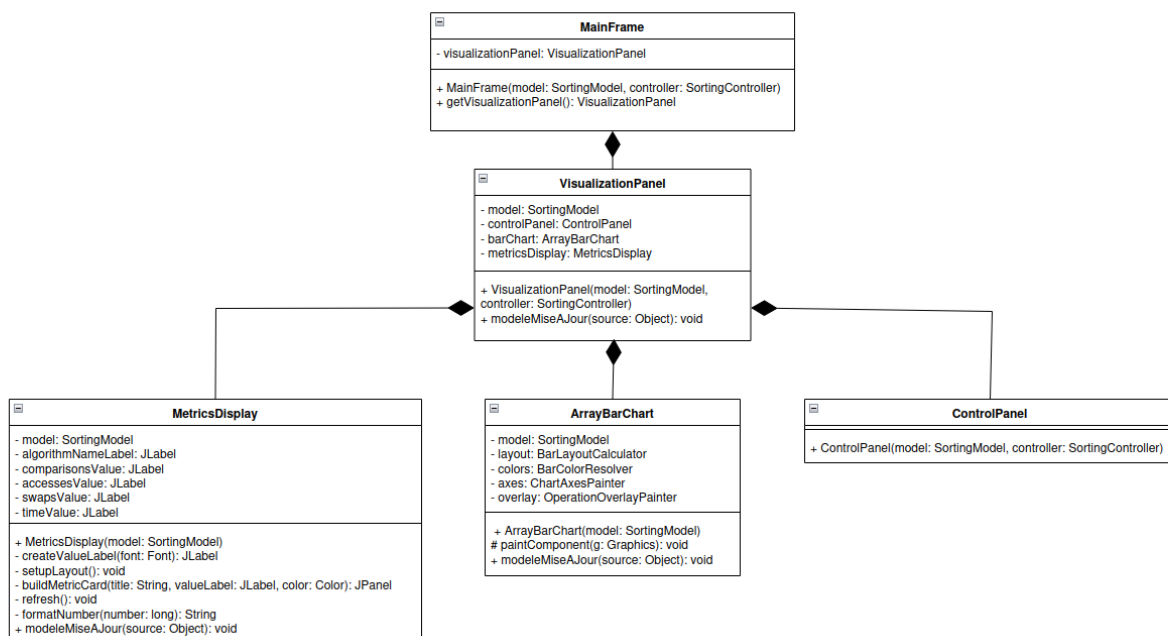


FIGURE 5 – Composition de la vue : MainFrame et ses sous-composants

4.4.2 Graphique en barres : ArrayBarChart

`ArrayBarChart` délègue chaque responsabilité à une classe dédiée : `BarLayoutCalculator` calcule la géométrie de chaque barre, `BarColorResolver` résout la couleur selon l'opération en cours, `ChartAxesPainter` dessine les axes, et `OperationOverlayPainter` affiche le nom de l'opération. La couleur du texte est cohérente avec celle des barres grâce à la classe utilitaire partagée `OperationColors`.

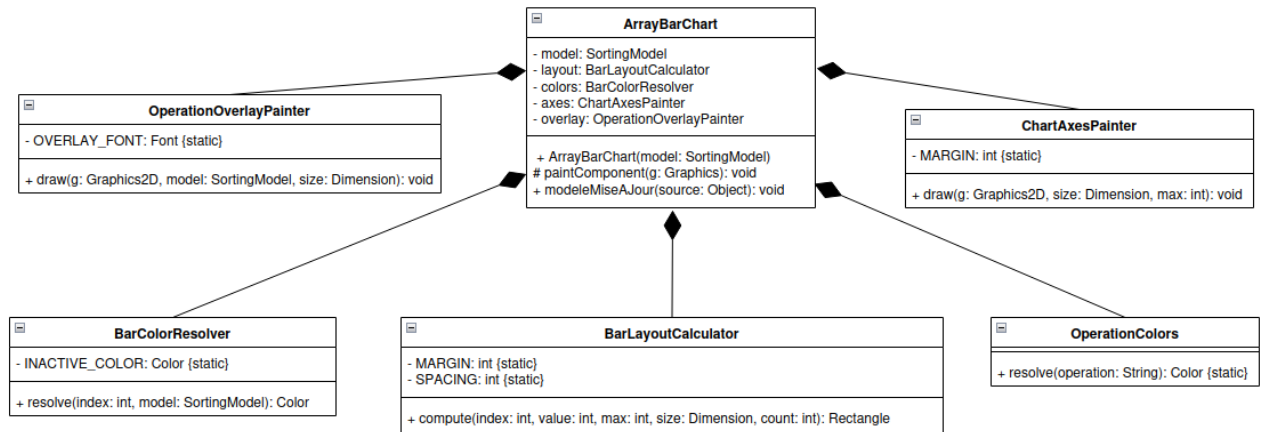


FIGURE 6 – Composition interne d'ArrayBarChart

4.4.3 Panneau de contrôle : ControlPanel

`ControlPanel` agrège quatre sous-panneaux spécialisés sans aucune logique propre. `ExecutionControlPanel` implémente une machine d'état à trois états (*repos*, *en tri*, *terminé*) via trois méthodes nommées `applyIdleState`, `applySortingState` et `applyFinishedState`, garantissant que les boutons reflètent toujours l'état réel de l'application.

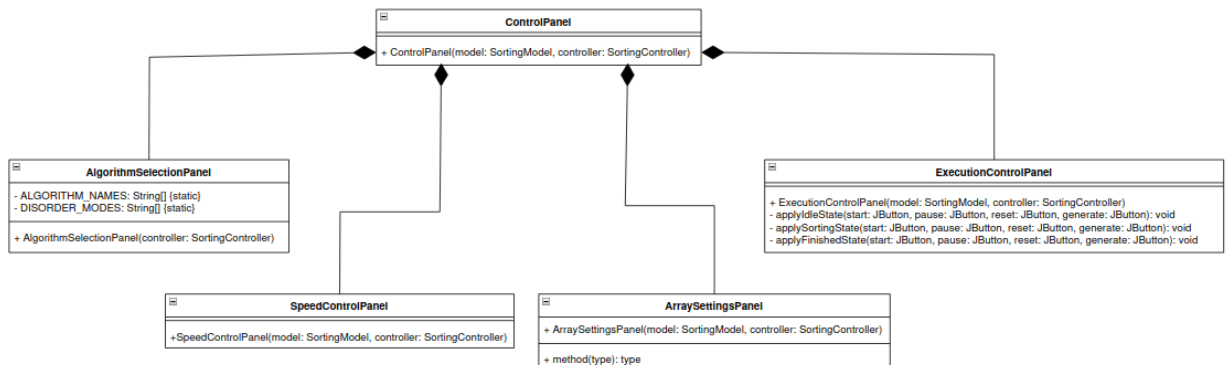


FIGURE 7 – Composition du ControlPanel et de ses sous-panneaux

5 Expérimentations et usages

Cette partie expérimentation est constituée en deux parties : La première a pour but de générer les données qui seront analysées plus tard. La seconde a pour but d'analyser les résultats obtenues pour démontrer le comportement des algorithmes de tri ou une tendance générale ou particulière.

5.1 Expérimentation

Dans cette première partie nous cherchons à générer un nombre de données exhaustives pour pouvoir analyser le comportement des algorithmes de tri selon différents paramètres d'entrée.

5.1.1 Méthode de fonctionnement

L'expérimentation repose sur une architecture composée de trois couches distinctes :

Couche d'orchestration (Bash) : Les scripts shell contrôlent l'ensemble du processus d'expérimentation. Le script principal `Lanceur.sh` accepte trois modes d'exécution : **expe** (expérimentation seule), **aggr** (agrégation seule), ou **tout** (les deux séquentiellement). Cette approche permet de relancer l'agrégation sur des données existantes sans refaire les calculs coûteux.

Couche d'exécution (Java) : La classe `Experimentation` constitue le cœur du protocole d'expérimentation. Contrairement à la version interactive, elle exécute 100 répétitions consécutives pour chaque combinaison de paramètres afin d'obtenir des mesures statistiquement significatives. L'optimisation principale réside dans la parallélisation des cinq algorithmes via des threads Java natifs : chaque tri s'exécute sur sa propre copie du tableau dans un thread dédié, réduisant considérablement le temps total d'expérimentation.

Couche de stockage (CSV) : Les résultats sont directement exportés au format CSV via des balises `[csv]` dans la sortie standard, puis capturés et agrégés par les scripts Bash. Cette approche évite les dépendances externes et garantit la compatibilité avec les outils d'analyse standard.

5.1.2 Lancement de l'expérimentation

L'expérimentation suit trois étapes orchestrées par `Lanceur.sh` :

```
Lanceur.sh [dossier] [mode]
L Mode "expe" ou "tout"
|   L- Compilation automatique (ant compile)
|   L-- Pour chaque taille {100, 500, 1000, 5000, 10000, 25000, 50000, 75000, 100000}
|       L-- experimentation.sh [taille] [dossier]
|           L-- Pour chaque (désordre, type) {0...100%} × {1,2,3,4}
|               L-- java Experimentation [taille] [désordre] [type]
L-- Mode "aggr" ou "tout"
    L-- Aggregation.sh [source] [destination]
```

Le paramétrage expérimental couvre :

- **Tailles** : 9 valeurs de 100 à 100 000 éléments
- **Désordre** : 11 niveaux de 0% à 100% par pas de 10%
- **Types** : 4 localisations (aléatoire, début, milieu, fin)
- **Répétitions** : 100 exécutions par combinaison

Cela permet d'avoir un grand nombre de mesures variées.

5.1.3 Génération des résultats

Les résultats sont organisés par taille dans des fichiers CSV dédiés (`size_100.csv`, `size_500.csv`, etc.) avec la structure suivante :

Listing 3 – Structure des fichiers CSV de résultats

```

taille ; desordre ; typeDesordre ; algo ; acces ; comparaisons ; echanges ; temps
100;0;1;QuickSort;1234;567;89;123456
100;0;1;InsertionSort;297;99;0;11700
...
```

Chaque ligne représente l'exécution d'un algorithme sur une instance donnée.

La parallélisation Java réduit significativement le temps d'expérimentation : au lieu d'exécuter 500 tris séquentiellement par combinaison de paramètres (100 répétitions $\times$ 5 algorithmes), les cinq algorithmes s'exécutent simultanément, divisant théoriquement le temps par 5.

5.1.4 Aggregation des résultats

Une fois les expérimentations terminées, la phase d'agrégation réorganise les données par algorithme via le script `Aggregation.sh`. Cette étape utilise `awk` pour filtrer les lignes de chaque fichier `size_*.csv` selon l'algorithme et les consolide dans des fichiers dédiés :

```

resultats/Experimentation_1/
L-- size_100.csv      # Données brutes par taille
L-- size_500.csv
L-- ...
L-- par_algo/        # Données agrégées par algorithme
  L-- QuickSort.csv
  L-- InsertionSort.csv
  L- MergeSort.csv
  L-- BubbleSort.csv
  L- CountingSort.csv
```

Cette double organisation facilite deux types d'analyses complémentaires : l'étude de l'impact de la taille (fichiers `size_*.csv`) et la comparaison entre les algorithmes sur l'ensemble des configurations (fichiers `par_algo/*.csv`). L'étape d'agrégation peut être relancée indépendamment via `bash Lanceur.sh [dossier] aggr`, permettant de restructurer les données sans refaire les calculs coûteux.

Cette architecture modulaire garantit la reproductibilité des résultats tout en optimisant les temps de calcul grâce à la parallélisation des algorithmes de tri.

5.2 Analyse des résultats

Dans cette partie nous cherchons à utiliser les données créées dans la partie 5.1 pour pouvoir les transformer en courbe pour faciliter leur analyse.

5.2.1 Lecture et traitement des données

Les résultats bruts des expérimentations sont stockés dans des fichiers CSV, un par algorithme, dont le chemin est `resultats/Experimentation_2/algos/<Algorithme>.csv`. Chaque ligne représente une exécution individuelle et contient huit champs : la taille du tableau (**size**), le taux de désordre (**disorder**), le type de localisation du désordre (**type**), le nom de l'algorithme (**algo**), ainsi que les quatre métriques mesurées - accès mémoire, comparaisons, swaps et temps d'exécution.

```

data = defaultdict(lambda: {
    "access": [], "compare": [], "swap": [], "time": []
})

for algo in algorithms:
    with open(f"{base_path}/{algo}.csv") as f:
```

```

reader = csv.reader(f, delimiter=';')
next(reader) # skip header du fichier csv
for row in reader:
    size, disorder, dtype, algo_name = (
        int(row[0]), int(row[1]), row[2], row[3]
    )
    key = (algo_name, size, disorder, dtype)
    data[key]["access"].append(int(row[4]))
    data[key]["compare"].append(int(row[5]))
    data[key]["swap"].append(int(row[6]))
    data[key]["time"].append(int(row[7]))

```

Listing 4 – Lecture des fichiers CSV et agrégation des mesures

Chaque combinaison (algorithme, taille, désordre, type) est exécuter une dizaine de fois, les valeurs sont accumulées dans des listes. La moyenne est ensuite calculée pour chaque combinaison afin de lisser les variations dues au bruit de mesure :

```

mean_data = {}
for key, metrics in data.items():
    mean_data[key] = {
        "access": np.mean(metrics["access"]),
        "compare": np.mean(metrics["compare"]),
        "swap": np.mean(metrics["swap"]),
        "time": np.mean(metrics["time"])
    }

```

Listing 5 – Calcul des moyennes par combinaison

5.2.2 Génération des courbes

Trois familles de graphiques sont générées à partir du dictionnaire `mean_data` :

- La première famille, produit pour chaque combinaison (taille, type, métrique) une figure où chaque algorithme est représenté par une courbe en fonction du taux de désordre. Elle permet d'évaluer directement la hiérarchie des algorithmes sur une même métrique. L'échelle logarithmique n'a été employée que pour les courbes de temps d'exécution. Les autres métriques présentent des valeurs nulles, ce qui interdit l'usage d'une échelle logarithmique, celle-ci ne pouvant représenter la valeur 0. Une échelle linéaire a donc été retenue pour ces mesures.
- La seconde famille, produit pour chaque combinaison (algorithme, taille, métrique) une figure où chaque type de localisation du désordre est représenté par une courbe distincte. Elle permet de constater l'effet de la répartition du désordre sur une même combinaisons algorithme, taille, métrique
- Une troisième famille de courbes, reprenant les codes de la première, exclut BubbleSort et InsertionSort de la comparaison inter-algorithmes. Cette version filtrée utilise la liste `algorithms_filter`. Elle a été introduite pour pallier l'écrasement visuel des algorithmes efficaces par les algorithmes $O(n^2)$ sur les graphes complets, et plusieurs figures issues de cette famille sont utilisées dans l'analyse présentée à la section suivante.

L'ensemble des courbes générées - soit 5 algorithmes $\times$ 3 tailles $\times$ 4 types $\times$ 4 métriques figures pour la première famille, et autant pour la seconde - est mis à disposition en annexe. Seules les figures les plus représentatives sont commentées dans le corps du rapport, le lecteur étant invité à consulter l'archives zip en annexe pour les configurations non détaillées.

6 Réponse à la problématique

Cette étude a pour objectif de répondre à la question suivante : **Quelle est l'efficacité des algorithmes de tri en fonction du désordre, tant en quantité qu'en répartition ?** Cinq algorithmes ont été évalués - BubbleSort, InsertionSort, MergeSort, QuickSort et CountingSort - sur trois tailles de tableau ($n = 10\,000$, $50\,000$, $100\,000$), avec quatre types de localisation du désordre (type 1 : généralisé, type 2 : début, type 3 : milieu, type 4 : fin) et un taux de désordre variant de 0% à 100% avec une incrémentation de 10%. Quatre métriques ont été mesurées : le nombre de comparaisons, le nombre d'accès mémoire, le nombre de swaps et le temps d'exécution. L'ensemble des courbes générées est disponible en annexe. Lorsqu'un schéma se répète à travers les différentes tailles ou types, une courbe représentative est présentée et la généralisation est explicitée.

6.1 Comparaisons

6.1.1 Hiérarchie générale

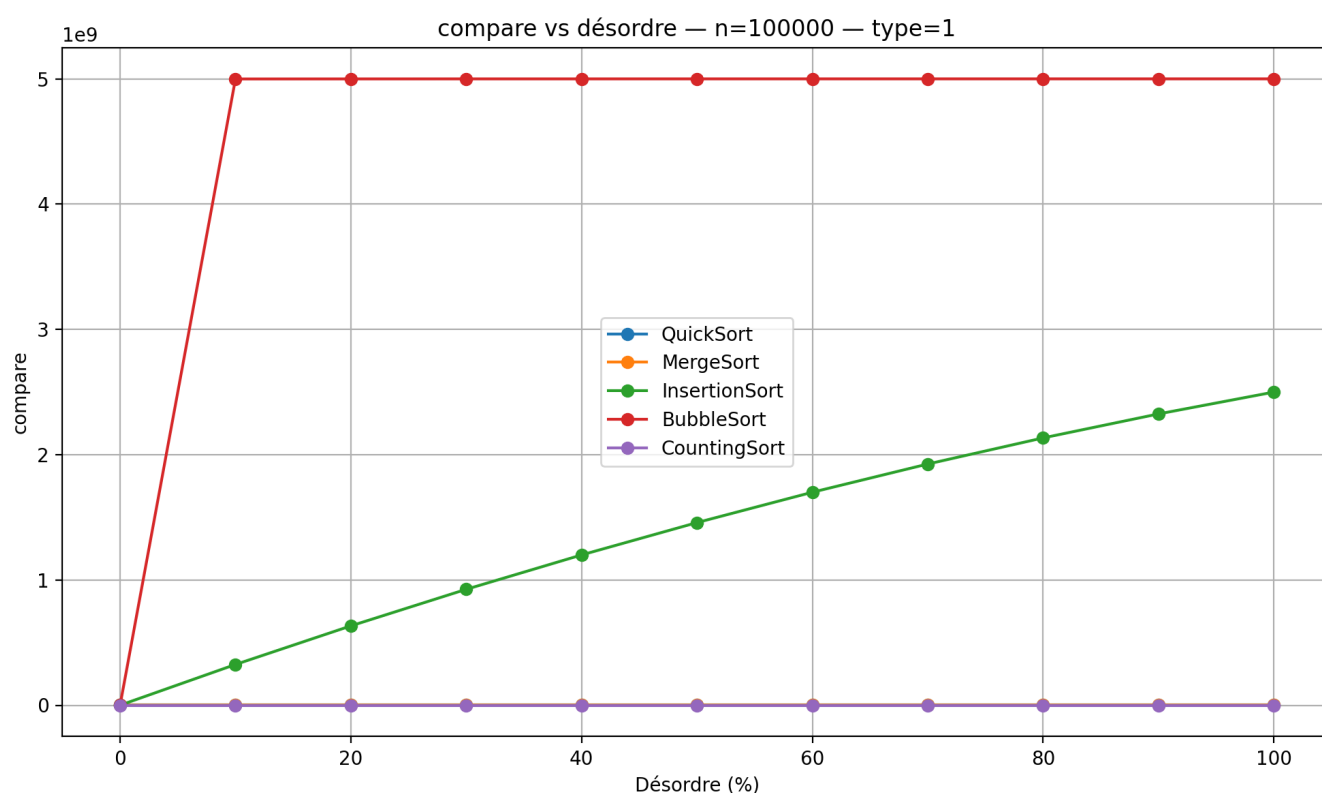


FIGURE 8

La Figure 8 illustre de manière saisissante la différence de complexité entre les algorithmes. BubbleSort et InsertionSort, tous deux $O(n^2)$, dominent largement le graphe et rendent les trois autres algorithmes quasiment invisibles à cette échelle. Ce constat se vérifie à toutes les tailles testées. Pour révéler le comportement des algorithmes efficaces, la Figure 9 présentent les mêmes données en excluant BubbleSort et InsertionSort.

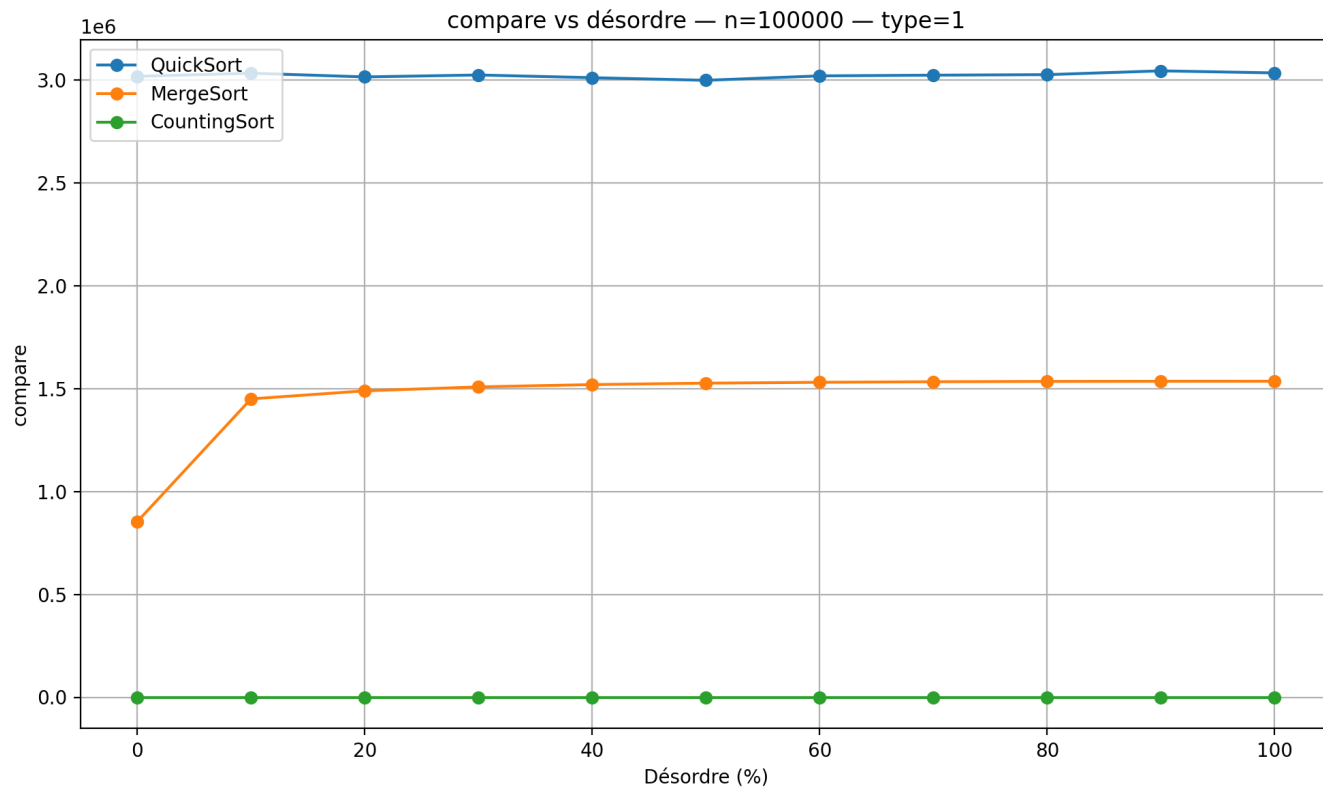


FIGURE 9

On observe que CountingSort maintient un nombre de comparaisons strictement nul, ce qui est attendu d'un algorithme non-comparatif. MergeSort présente un comportement quasi-constant, légèrement croissant avec le désordre. QuickSort fluctue autour d'une valeur stable indépendamment du taux de désordre, traduisant l'effet du pivot aléatoire.

6.1.2 Influence de la localisation du désordre

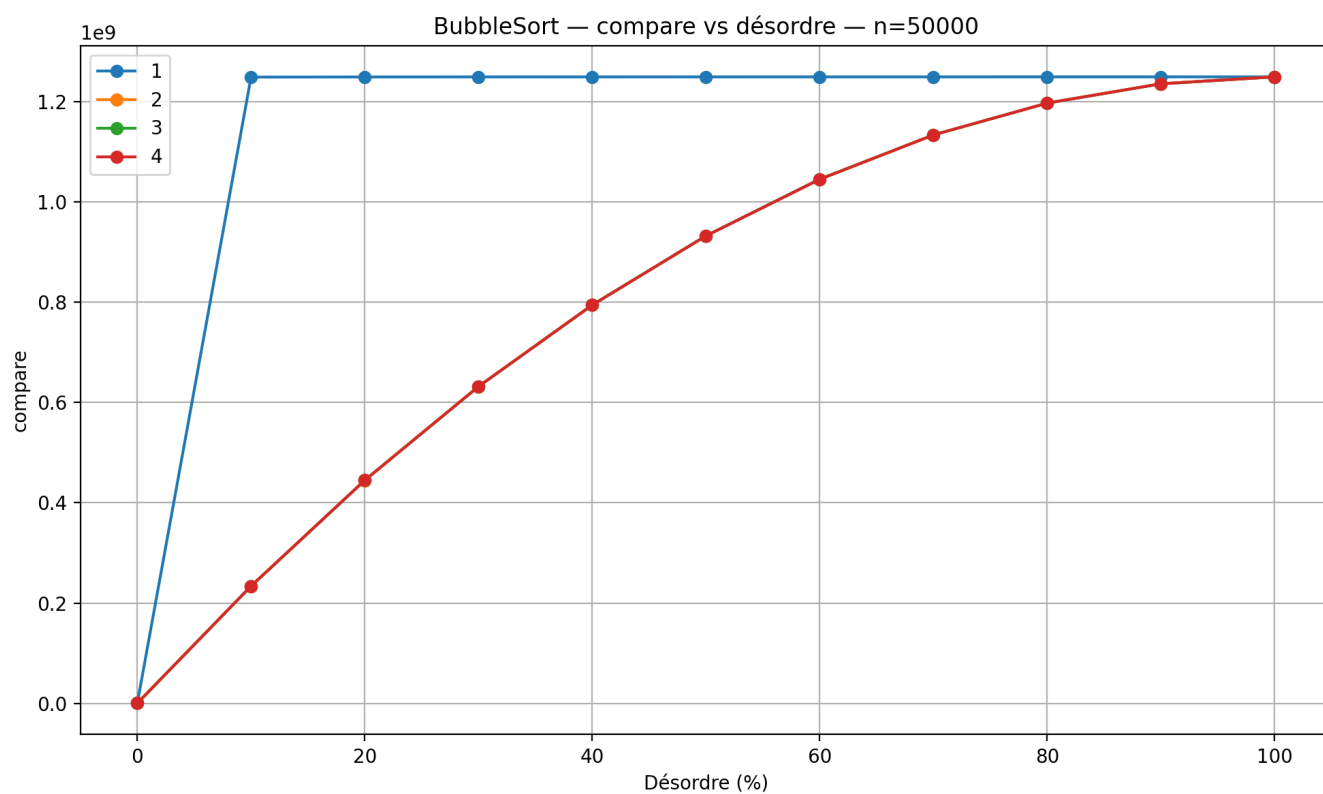


FIGURE 10

La Figure 10 met en évidence l'impact de la localisation du désordre sur BubbleSort. Le type 1 (désordre généralisé) sature le compteur de comparaisons dès 10% de désordre, puis se stabilise. Les types 2, 3 et 4 (désordre localisé en début, milieu ou fin) produisent une progression linéaire et convergent vers la même valeur maximale à 100% de désordre. Ce schéma se répète de façon identique pour MergeSort avec de valeur plus réduite et est observable sur l'ensemble des tailles testées. Pour QuickSort en revanche, la localisation du désordre n'a aucun effet mesurable.

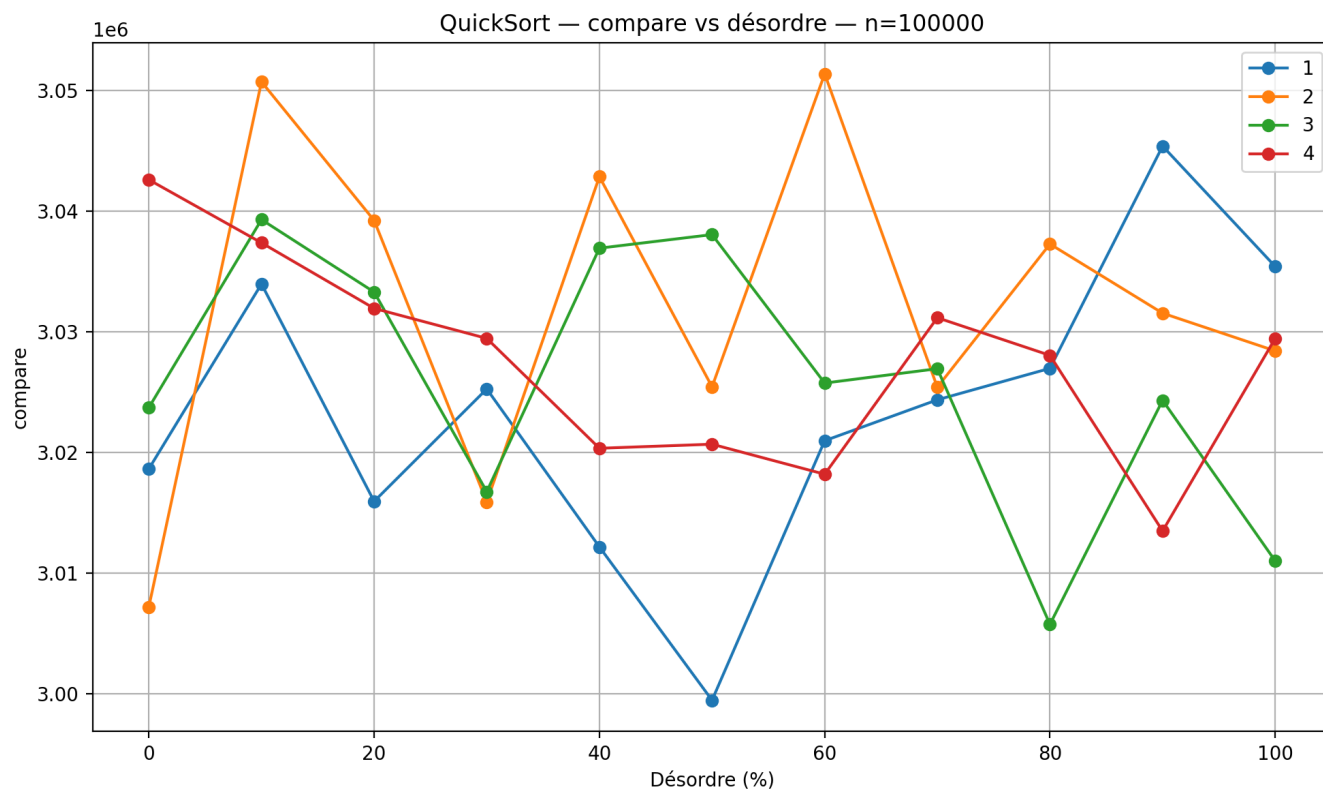


FIGURE 11

La Figure 11 confirme que les quatre types de désordre produisent des courbes indiscernables, fluctuant autour d'une valeur constante. Ce comportement, stable quelle que soit la taille du tableau, est caractéristique d'une implémentation avec un pivot aléatoire qui neutralise l'influence de l'ordre initial des données.

6.2 Accès

6.2.1 Hiérarchie générale

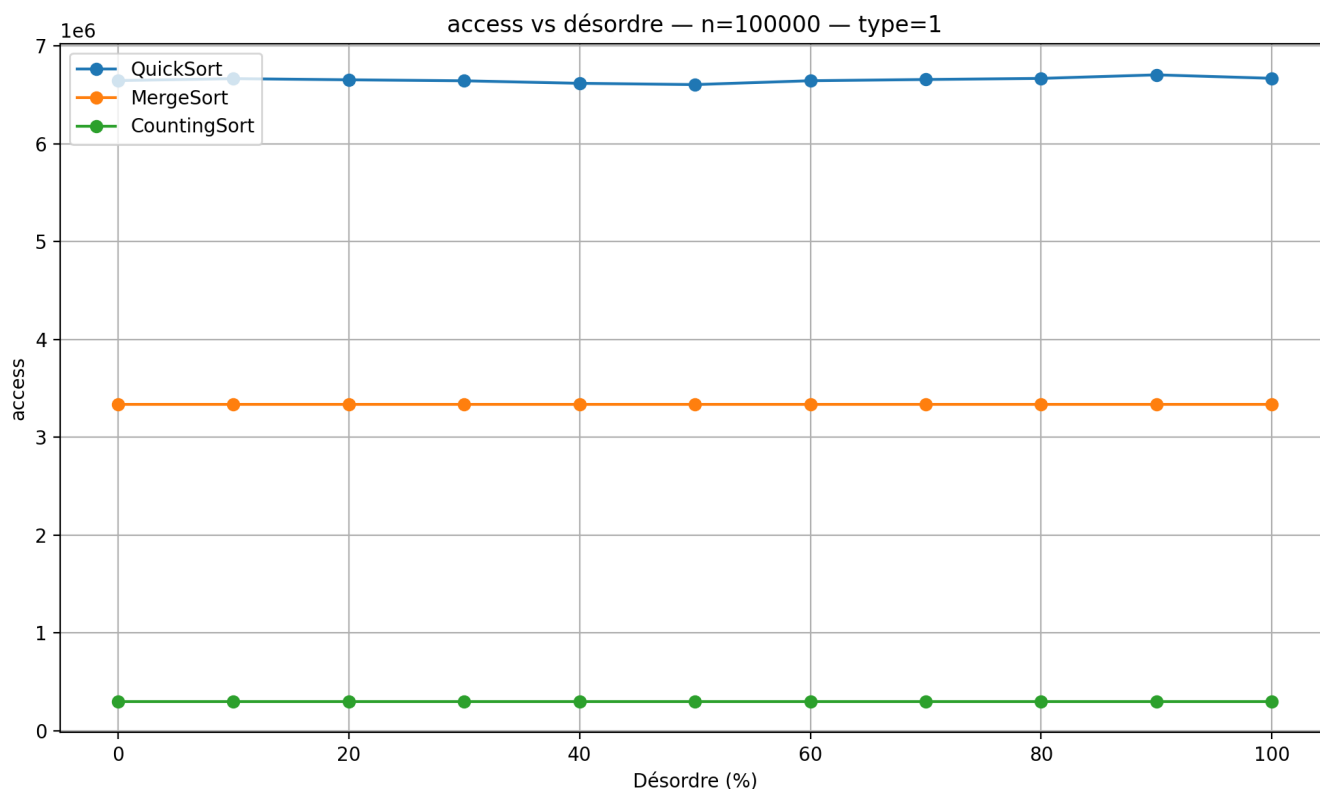


FIGURE 12

Les accès mémoire reflètent globalement la même hiérarchie que les comparaisons. BubbleSort génère environ deux fois plus d'accès qu'InsertionSort, qui lui-même dépasse largement les trois autres algorithmes - ces derniers étant, là encore, invisibles sur la version non filtrée des graphes. La Figure 12 révèle le comportement des algorithmes efficaces. MergeSort présente un nombre d'accès rigoureusement constant quel que soit le taux ou le type de désordre, ce qui s'explique par sa structure de fusion systématique indépendante de l'ordre des données. QuickSort exhibe de légères fluctuations autour d'une valeur stable, cohérentes avec ses variations de comparaisons. CountingSort maintient un nombre d'accès constant de l'ordre de $O(n + k)$, où k représente la plage de valeurs des données - ce nombre est donc indépendant du désordre mais dépendant des données elles-mêmes.

6.2.2 Influence de la localisation du désordre

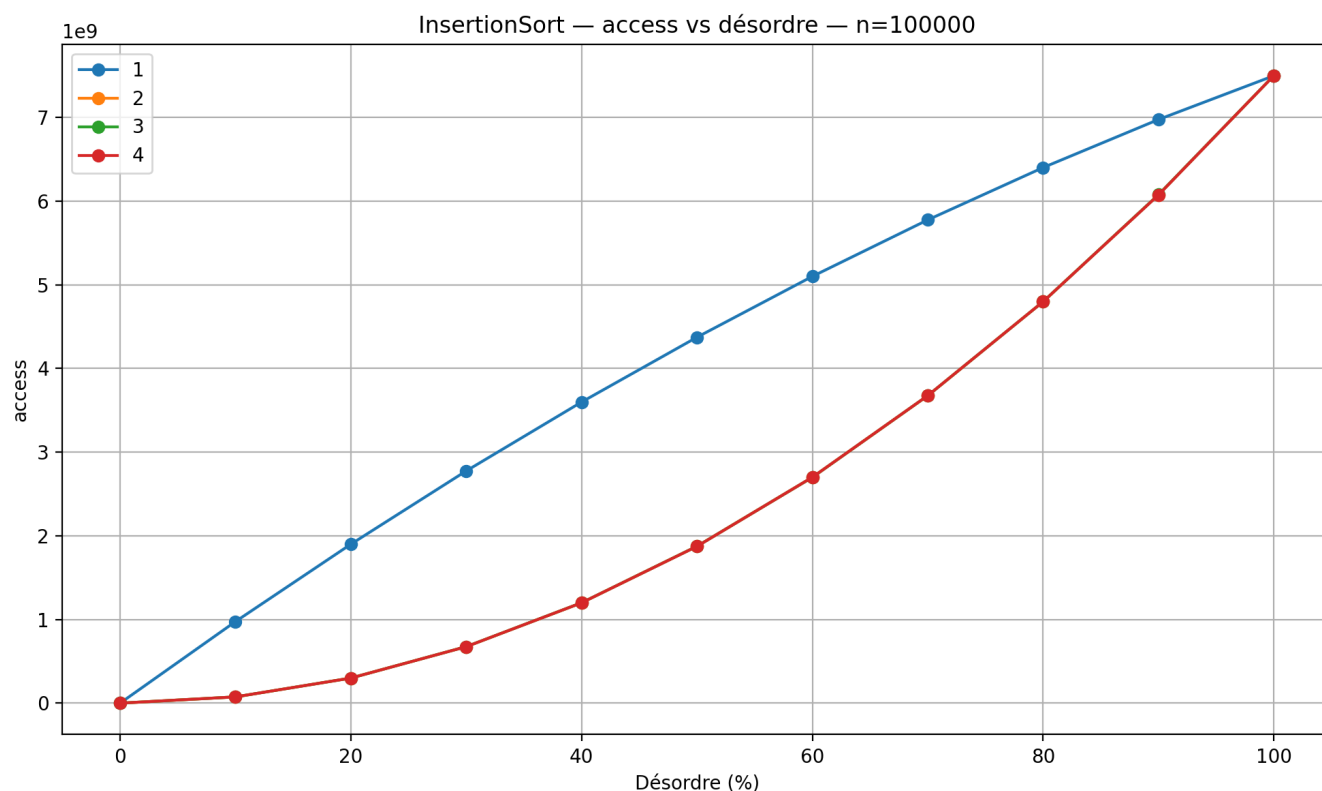


FIGURE 13

La Figure 13 illustre l'influence de la localisation du désordre sur les accès mémoire d'InsertionSort. Le type 1 (désordre généralisé) induit une croissance quasi-linéaire dès les premiers pourcents de désordre - chaque élément désordonné, où qu'il se trouve, provoque immédiatement des décalages coûteux en accès. Le type 2, 3, 4 (désordre localisé) présente une courbe concave qui décolle tardivement. La superposition des courbes suggère que la position exacte du désordre - début, milieu, fin - n'introduit pas de différence mesurable dès lors qu'il est localisé. BubbleSort suit plus ou moins le même schéma sur cette métrique, cohérent avec les observations faites sur les comparaisons en section 2.3. Pour les algorithmes efficaces, la localisation reste sans effet mesurable.

6.3 Swap

6.3.1 Hiérarchie générale

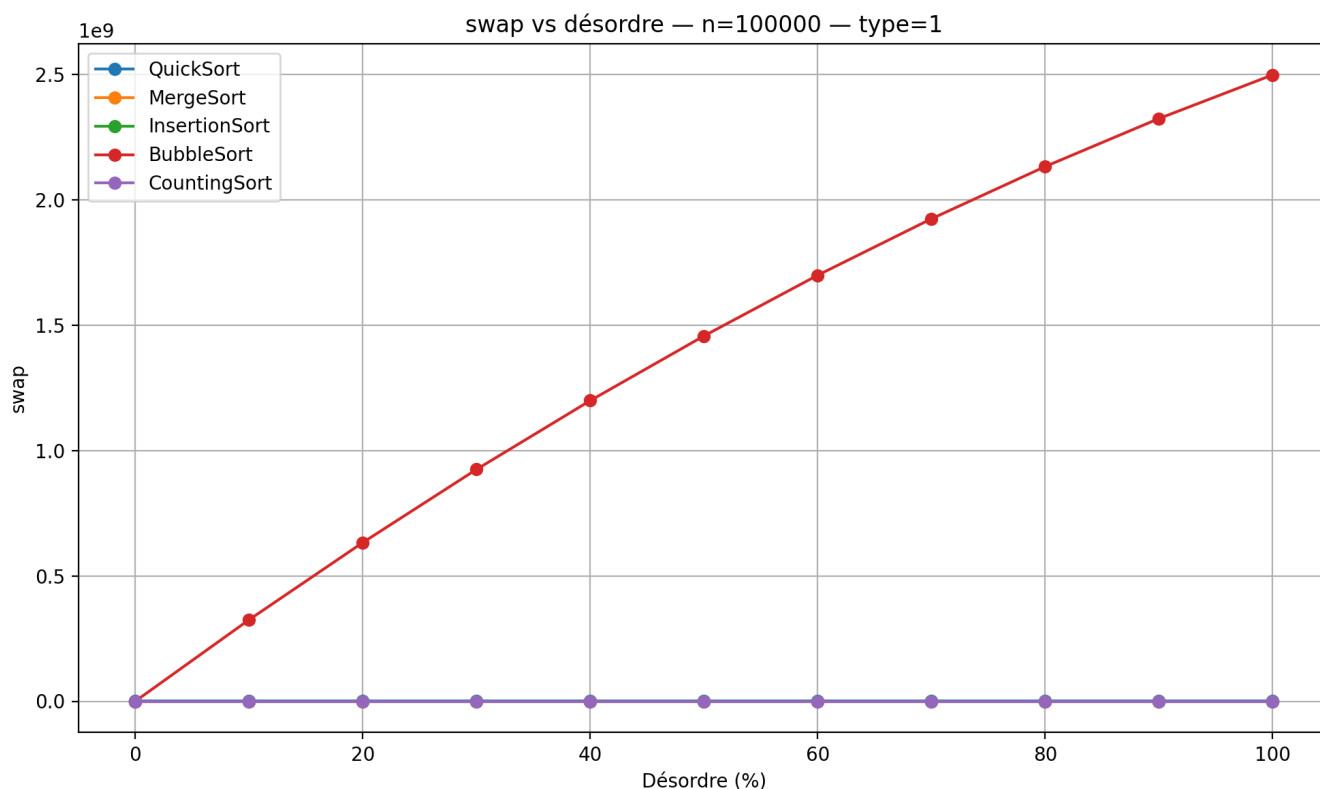


FIGURE 14

La Figure 14 résume à elle seule l'intégralité de la section swaps. BubbleSort est le seul algorithme à générer des échanges en quantité significative - les quatre autres affichent une valeur nulle ou négligeable à cette échelle. Ce résultat s'explique par des raisons structurelles différentes selon les algorithmes : MergeSort et CountingSort ne procèdent jamais par échanges en place, InsertionSort opère par décalages et non par swaps, et QuickSort, bien qu'il effectue des échanges réels, en génère un nombre négligeable comparé à BubbleSort. Chez BubbleSort, le type 1 produit une croissance linéaire des swaps avec le désordre, tandis que les types localisés 2, 3 et 4 présentent une croissance quadratique.

6.4 Temps d'exécution

6.4.1 Hiérarchie générale

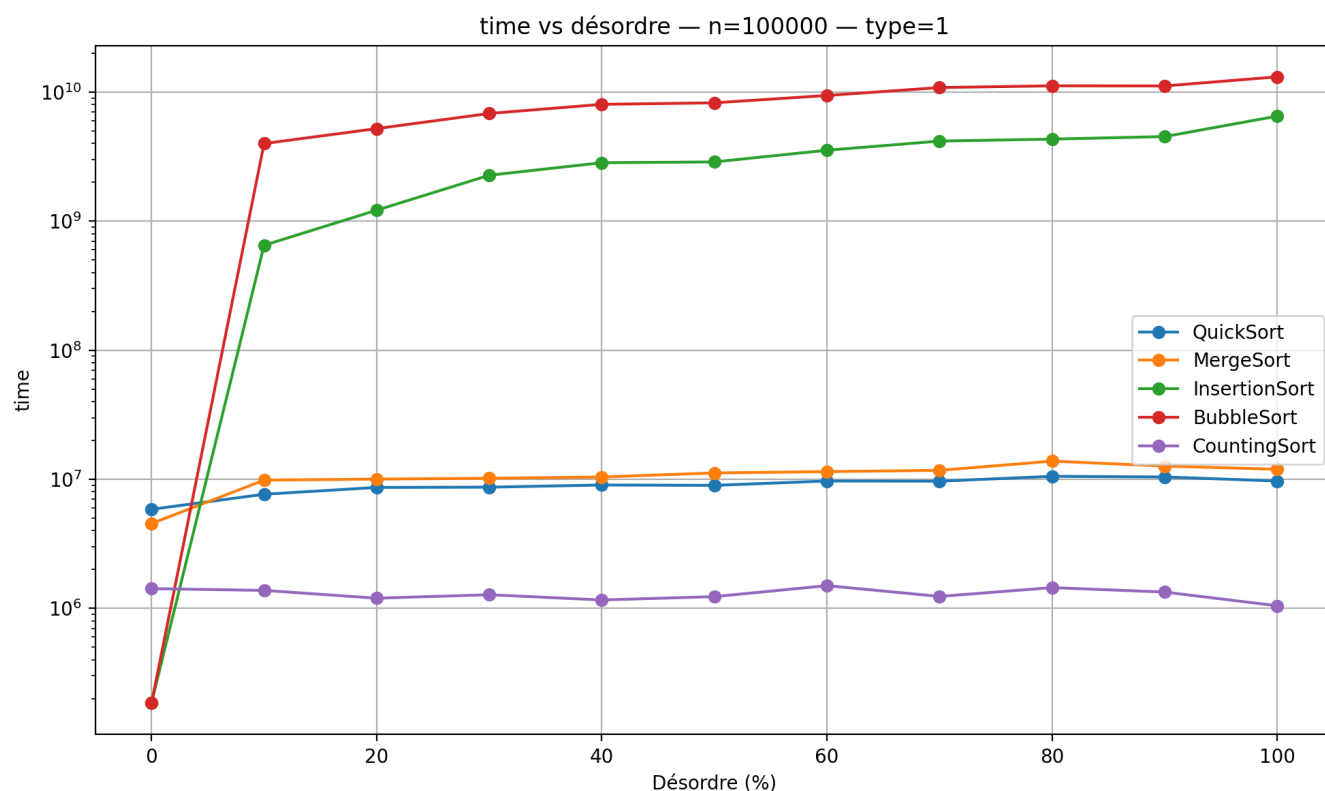


FIGURE 15

Le temps d'exécution constitue la métrique synthétique qui intègre l'ensemble des opérations précédentes. La Figure 15, présentée en échelle logarithmique pour rendre lisibles tous les algorithmes simultanément, confirme la hiérarchie observée sur les autres métriques. BubbleSort et InsertionSort sont systématiquement les plus lents, quelle que soit la configuration testée. MergeSort, malgré sa complexité théorique $O(n \log n)$, présente des temps comparables voire supérieurs à InsertionSort dans certaines configurations de faible désordre, en raison de l'overhead lié à l'allocation de tableaux temporaires lors des fusions. QuickSort s'impose comme l'algorithme le plus rapide et le plus stable parmi les algorithmes comparatifs, avec des temps quasi-constants indépendamment du type et du taux de désordre. CountingSort présente des temps d'exécution qui semblent erratiques à première vue. Son comportement réel est $O(n + k)$ et strictement indépendant du désordre, comme le confirment ses métriques de comparaisons et d'accès.

6.5 Synthèse par algorithme

Algorithme	Comparaisons	Swap	Accès	Temps	Sensibilité au désordre
BubbleSort	Très élevé	Très élevé	Très élevé	Très lent	Forte
InsertionSort	Élevé	0	Élevé	Lent	Modérée
QuickSort	Faible	Faible	Faible	Moyen	Faible
MergeSort	Faible	0	Faible, constant	Moyen	Modérée
CountingSort	0	0	constant	Rapide	Nulle

TABLE 3 – Comparaison des algorithmes implémentés

- Données générales, sans contrainte particulière sur la structure ou le désordre :
QuickSort apparaît comme l'algorithme le plus efficace : rapide, stable en pratique et peu sensible à la quantité ou à la localisation du désordre.
- Besoin de stabilité ou de comportement strictement déterministe
MergeSort constitue une alternative robuste. Son coût reste légèrement supérieur à celui de QuickSort, mais il offre une prédictibilité totale et n'effectue aucun échange en place.
- Tableaux de petite taille ou faiblement désordonnés
InsertionSort peut être compétitif : son coût devient quasi linéaire lorsque le désordre est faible, ce qui en fait un bon candidat pour des données presque triées.
- Données entières appartenant à une plage de valeurs connue
CountingSort surpasse largement les algorithmes comparatifs. Son temps d'exécution, indépendant du désordre, en fait la solution la plus performante dans ce cadre spécifique.
- BubbleSort
Les résultats confirment que BubbleSort n'est adapté à aucun usage pratique : sa sensibilité au désordre et sa complexité quadratique le rendent systématiquement non compétitif.

6.6 Conclusion

L'analyse confirme que la complexité algorithmique théorique se traduit de manière directe et mesurable dans les données expérimentales. Les algorithmes $O(n^2)$ - BubbleSort et InsertionSort - s'avèrent inutilisables à grande échelle, avec des métriques plusieurs ordres de grandeur supérieures aux algorithmes $O(n \log n)$. Parmi les algorithmes efficaces, QuickSort se distingue comme le choix généraliste optimal : rapide, stable, et remarquablement insensible à la quantité comme à la localisation du désordre. MergeSort offre une alternative prévisible et sans swaps, au prix d'une utilisation plus importante de la mémoire. CountingSort surpasse tous les autres dans son domaine d'application, mais reste conditionné à des données entières dans une plage connue. La localisation du désordre s'avère un facteur secondaire mais non négligeable : elle influence significativement les algorithmes $O(n^2)$, notamment BubbleSort tandis qu'elle reste sans effet mesurable sur QuickSort et MergeSort.